

Reviewer Pack — Fourier Coefficient Sum Submodularity for Monotone DNF

Entry d5d05615bf79 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 17:37:17 UTC

Fourier Coefficient Sum Submodularity for Monotone DNF

Entry ID: d5d05615bf79 Verdict: INCONCLUSIVE Recorded: 2026-05-09 17:37:17 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis on Boolean Functions **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

For any monotone DNF formula f on n variables, let $\mu(f) = \sum_{\{S \subseteq [n], |S| \leq \log n\}} |\hat{f}(S)|$. Then $\mu(f \wedge g) \geq \mu(f) + \mu(g) - \mu(f \vee g)$ for all monotone DNF f, g , and $\mu(k\text{-CLIQUE}) = \Omega(n)$.

Rationale (proposer’s reasoning):

Submodularity of μ under conjunction captures trade-offs in variable interactions, while the $\log n$ cutoff balances expressiveness and complexity. The $k\text{-CLIQUE}$ lower bound stems from its high sensitivity to dense subgraphs, which amplifies Fourier coefficients at large subsets.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b1b62b949fc252e2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def fast_walsh_hadamard_transform(f):
    n = len(f)
    if n == 1:
        return f
    even = fast_walsh_hadamard_transform(f[0::2])
    odd = fast_walsh_hadamard_transform(f[1::2])
    result = [0] * n
    for k in range(n // 2):
        result[k] = even[k] + odd[k]
        result[k + n // 2] = even[k] - odd[k]
    return result

def walsh_hadamard_transform(f):
    n = len(f)
    while n > 1:
        f = fast_walsh_hadamard_transform(f)
        n //= 2
    return f

def fourier_coefficient_sum(f, log_n):
    transform = walsh_hadamard_transform(f)
    mu = sum(abs(coeff) for coeff in transform[:2**log_n])
    return mu

def generate_monotone_dnf(n):
    num_clauses = random.randint(1, n // 2)
    clauses = []
    for _ in range(num_clauses):
        clause = [random.choice([0, 1]) * (i + 1) for i in range(n)]
        if all(clause[i] == 0 or clause[i] == clause[0] for i in range(1, n)):
            clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    log_n = math.ceil(math.log2(n))

    f = generate_monotone_dnf(n)
    g = generate_monotone_dnf(n)

    mu_f = fourier_coefficient_sum(f, log_n)
```

```

mu_g = fourier_coefficient_sum(g, log_n)
mu_fg = fourier_coefficient_sum([x and y for x in f for y in g], log_n)
mu_fog = fourier_coefficient_sum([x or y for x in f for y in g], log_n)

conjecture_holds = (mu_fg >= mu_f + mu_g - mu_fog)
counterexample = "" if conjecture_holds else "counterexample"

return {
    "metric_name": " $\mu(f) + \mu(g) - \mu(f \vee g)$ ",
    "metric_value": mu_fg - mu_f - mu_g + mu_fog,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

IAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}

```

TRIAL: {'metric_name': ' $\mu(f) + \mu(g) - \mu(f \vee g)$ ', 'metric_value': 0, 'instances_tested': 1, 'conjecture': 1}
RESULT: SUPPORTED mean=0.0 std=0.0 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111255
2	propose	ollama_remote	qwen3:8b	0	0	92076
3	preregistration	ollama_remote	qwen3:8b	0	0	24245
4	novelty	ollama_remote	qwen3:8b	0	0	20731
5	novelty	ollama_remote	qwen3:8b	0	0	16914
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12196
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9770
8	critic	ollama_remote	qwen3:8b	0	0	30056
9	judge	ollama_remote	qwen3:8b	0	0	21125

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 338368 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d5d05615bf79.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d5d05615bf79.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d5d05615bf79.tar.gz` (if generated)